THE APPLIED AI ENGINEER

A Production-Ready Guide to GenAI & Data Science

Chapter 4

Retrieval Augmented Generation (RAG) with Azure AI Search

What You Will Learn

- ✦ The RAG architecture and why it outperforms naive LLM prompting
- ✦ Azure AI Search: vector, full-text, and hybrid retrieval
- ✦ Embeddings, HNSW indexing, and BM25 scoring demystified
- ✦ Semantic Reranker: intent-aware result promotion
- ✦ End-to-end Python notebook: Hotel Reviews RAG pipeline
- ✦ Every code cell explained for production readiness

Prerequisites: Chapter 3 · Python 3.8+ · Azure Subscription · OpenAI API Key

01 Theory — Understanding RAG & Azure AI Search

1. The Problem RAG Solves

Large Language Models (LLMs) like GPT-4 are extraordinarily capable — they can reason, write, summarise, and code. But they have a fundamental constraint: their knowledge is frozen at training time. If you ask an LLM about your company's internal hotel reviews, last week's customer complaints, or proprietary product specifications, it has nothing to draw from. It will either refuse to answer or, worse, confidently hallucinate plausible-sounding but entirely fabricated responses.

Retrieval Augmented Generation (RAG) is the architectural pattern that solves this. Instead of expecting the LLM to know everything, RAG gives it a dynamic, real-time information retrieval system. The LLM becomes the reasoning engine; the search index becomes its up-to-date knowledge base.

> **The Core Insight**
> RAG = Retrieval (find the right documents) + Augmented (inject them into context) + Generation (LLM produces a grounded answer). The LLM never answers from memory alone — it always grounds its response in retrieved evidence.

1.1 The RAG Architecture — End to End

A production RAG system has two distinct phases: the Indexing Phase (offline, run once or periodically) and the Query Phase (online, runs on every user request).

Indexing Phase (Offline Pipeline)

Stage	What Happens
1. Data Ingestion	Raw data (CSV, PDF, database rows, web pages) is loaded and cleaned. For our hotel reviews example: a Kaggle CSV of hotel reviews is downloaded.
2. Chunking	Long documents are split into smaller, semantically coherent chunks (typically 256–1024 tokens). Each chunk will become one searchable unit.
3. Embedding	Each chunk is passed through an embedding model (e.g., text-embedding-3-small from OpenAI) which converts it into a dense numerical vector — a list of 1,536 floats that encodes semantic meaning.

4. Index Storage	Each chunk's text, metadata (hotel name, city, date), and embedding vector are uploaded to Azure AI Search, which stores them in a specialised vector index.

Query Phase (Online, Per-User Request)

Stage	What Happens
1. Query Embedding	The user's natural-language query (e.g., 'hotels with great rooftop views in New York') is passed through the same embedding model to produce a query vector.
2. Retrieval	Azure AI Search compares the query vector against all stored document vectors and retrieves the top-k most semantically similar chunks. Optionally, keyword matching (BM25) and semantic reranking are also applied.
3. Context Assembly	The retrieved chunks are formatted into a structured prompt: system instructions + retrieved documents + user question.
4. LLM Generation	The assembled prompt is sent to the LLM (e.g., GPT-4o). The LLM generates a response grounded exclusively in the retrieved documents, with citations if configured.

1.2 Why Azure AI Search?

Azure AI Search (formerly Azure Cognitive Search) is Microsoft's enterprise-grade cloud search platform. It is one of the most complete RAG retrieval backends available because it supports all three retrieval paradigms in a single service:

Search Type	Mechanism	Best For
Full-Text (BM25)	Statistical term frequency / inverse document frequency scoring	Exact keyword matches, known terminology, filter queries
Vector Search (ANN)	Approximate Nearest Neighbour over embedding vectors using HNSW algorithm	Semantic meaning, paraphrase matching, intent understanding
Hybrid Search	RRF (Reciprocal Rank Fusion) merges BM25 and vector scores	Best of both worlds — most production systems use this
Semantic Reranker	Microsoft Bing-adapted deep learning model re-scores top results	High-precision use cases where top-1 accuracy is critical

Beyond retrieval, Azure AI Search provides: enterprise security (private endpoints, RBAC, encryption at rest), integrated data connectors (Azure Blob, SQL, Cosmos DB, SharePoint), built-in chunking and vectorisation pipelines (skillsets), and Azure OpenAI integration for in-index enrichment.

1.3 Key Concepts Demystified

1.3.1 Embeddings

An embedding is the mathematical representation of text as a point in high-dimensional space. The key property is that semantically similar texts are close to each other in this space, regardless of the exact words used.

Consider these three queries:

- "hotels with amazing pool facilities"
- "places to stay that have swimming"
- "accommodation with aquatic amenities"

All three mean the same thing. Traditional keyword search would score these as completely different queries (different words). An embedding model maps all three to nearby points in vector space, so a single review mentioning 'beautiful infinity pool' would be retrieved for all three queries. This is the superpower of semantic search.

Azure AI Search works with OpenAI's text-embedding-3-small (1,536 dimensions) and text-embedding-3-large (3,072 dimensions) out of the box, as well as any custom embedding model you choose to deploy.

1.3.2 HNSW — The Vector Index Algorithm

Storing one million embedding vectors and finding the nearest neighbour by brute force (comparing every query vector against every stored vector) would take seconds — unacceptable for a real-time search system. Azure AI Search uses HNSW (Hierarchical Navigable Small World) as its Approximate Nearest Neighbour (ANN) algorithm.

HNSW builds a layered graph structure where each node is a document embedding. At query time, the algorithm navigates this graph in milliseconds, finding vectors that are approximately (but extremely accurately) nearest to the query — trading a tiny bit of recall for massive speed gains. In practice, HNSW achieves 95–99% recall at millisecond latency even over millions of vectors.

HNSW Parameter	Effect
m (connections per node)	Higher = better recall but more memory. Typically 4–16.

efConstruction	Higher = better index quality but slower build time. Typically 400–800.
efSearch	Higher = better query recall but slower queries. Typically 500–1000.
vector_search_dimensions	Must match your embedding model's output size (e.g., 1536 for text-embedding-3-small).

1.3.3 BM25 — Full-Text Scoring

BM25 (Best Match 25) is the probabilistic information retrieval formula used by virtually every production search engine including Elasticsearch, Solr, and Azure AI Search. It scores documents based on:

- Term frequency (TF): how often does the query term appear in the document?
- Inverse document frequency (IDF): how rare is the query term across all documents? Rare terms get higher scores.
- Document length normalisation: shorter documents are not penalised for having a lower raw term count.

BM25 excels at precision for known-item searches ('find the review mentioning Hilton Times Square') but fails on semantic intent ('find reviews where guests felt safe and comfortable') — which is exactly where vector search steps in.

1.3.4 Hybrid Search with RRF

Hybrid search runs both BM25 and vector search simultaneously and merges the ranked result lists using Reciprocal Rank Fusion (RRF). RRF assigns each document a score of 1/(k + rank) for each list it appears in and sums them. This simple formula is remarkably effective at producing a combined ranking that outperforms either method alone — and is the default recommendation for production RAG systems.

1.3.5 Semantic Reranker

The Semantic Reranker is Azure AI Search's most powerful (and only paid-tier) feature. After BM25 or hybrid retrieval produces an initial result set (typically top 50 documents), the semantic reranker re-scores each document using a deep learning model trained on Bing's massive web search data.

This model understands the full contextual meaning of both the query and each document — not just token overlap. It also extracts two bonus outputs:

- Semantic Captions: the most relevant passage within each document, highlighted for the user
- Semantic Answers: if a direct answer to the query exists in any document, it is extracted and returned as a structured answer field

> **Pricing Note**
> The Semantic Reranker requires the Standard pricing tier or above on Azure AI Search. It is billed per 1,000 semantic queries. For most production RAG use cases, the accuracy improvement justifies the cost — especially for customer-facing applications where answer quality directly affects user satisfaction.

1.4 The PeerBNB Use Case — Grounding the Concepts

The scenario from your notes is an excellent, real-world grounding for these concepts. Let's articulate it precisely:

Business Requirement	Technical Translation
Guests should find hotels matching their personal preferences	Semantic intent matching — not keyword search
Recommendations should be based on what previous guests said	RAG over hotel review corpus, not LLM hallucination
'Even if there is no keyword matching'	Vector search over review embeddings
High-quality, relevant responses back to user	LLM generation grounded in top-k retrieved reviews
Scale to millions of reviews across global properties	Azure AI Search's enterprise-grade distributed index

The five-step implementation pipeline your notes outline — Data Preparation → Index Creation → Add Documents → Query Index → Semantic Reranking — maps directly to the RAG architecture described above. The rest of this chapter walks through each step in precise detail with production-ready Python code.

02 **Practical — Building the Hotel Reviews RAG Pipeline**

2. Environment Setup

Before writing any pipeline code, install all required dependencies and configure your environment variables. This project requires three Azure/OpenAI services: Azure AI Search (for the vector index), Azure OpenAI or OpenAI API (for embeddings), and optionally Azure OpenAI (for the generation step).

Cell 0 — Install Dependencies

```
# Cell 0: Install all required packages
# Run this once; restart the kernel afterward
# ──────────────────────────────────────────────────────────────────────
!pip install azure-search-documents==11.4.0      # Azure AI Search Python SDK
!pip install azure-core                           # Azure shared primitives
!pip install openai>=1.0.0                        # OpenAI Python SDK v1+
!pip install python-dotenv                        # .env file management
!pip install pandas                               # CSV handling
!pip install tqdm                                 # Progress bars for loops
```

Cell Explanation

We pin azure-search-documents==11.4.0 because this version introduced stable support for VectorSearch, SemanticConfiguration, and the SearchIndex management client — the three building blocks of our pipeline. The openai>=1.0.0 requirement is important: OpenAI's v1 SDK has a breaking API change from v0.x (no more openai.Embedding.create; use client.embeddings.create instead). Always pin major dependencies in production to avoid surprise breaking changes.

Cell 1 — Environment Configuration

```
# Cell 1: Load credentials from environment
# ──────────────────────────────────────────────────────────────────────
import os
from dotenv import load_dotenv

load_dotenv()  # Reads .env file from project root into os.environ

# Azure AI Search credentials
```

```
SEARCH_SERVICE_NAME = os.getenv('SERVICE_NAME')   # e.g. 'my-search-svc'
SEARCH_ADMIN_KEY    = os.getenv('SEARCH_ADMIN_KEY') # 52-char admin key
SEARCH_INDEX_NAME   = os.getenv('SEARCH_INDEX_NAME', 'hotel-reviews-index')

# Construct the full service endpoint URL
SEARCH_ENDPOINT = f'https://{SEARCH_SERVICE_NAME}.search.windows.net/'

# OpenAI credentials (for embeddings)
OPENAI_API_KEY      = os.getenv('OPENAI_API_KEY')
EMBEDDING_MODEL     = 'text-embedding-3-small'   # 1536-dim, cost-efficient
EMBEDDING_DIMS      = 1536

# Validate all credentials are present
required = [SEARCH_SERVICE_NAME, SEARCH_ADMIN_KEY, OPENAI_API_KEY]
assert all(required), 'One or more required environment variables are missing!'
print('All credentials loaded successfully ✓')
print(f'Search endpoint: {SEARCH_ENDPOINT}')
```

Cell Explanation

We use a consistent naming convention for environment variables (SCREAMING_SNAKE_CASE) and centralise all configuration at the top of the notebook. The assert statement provides an immediate, informative failure if any credential is missing — far better than a cryptic authentication error 10 cells later. The SEARCH_ENDPOINT format (https://{service_name}.search.windows.net/) is the standard pattern for all Azure AI Search REST calls. The .env file (never committed to Git) should contain: SERVICE_NAME, SEARCH_ADMIN_KEY, SEARCH_INDEX_NAME, and OPENAI_API_KEY.

3. Step 1 — Data Preparation: CSV to JSON

The hotel reviews dataset from Kaggle is a CSV file. Azure AI Search expects documents in JSON format, with each row becoming one JSON object (one searchable document). This step transforms raw CSV rows into individual JSON files, one per review, stored in a structured directory.

Cell 2 — Load and Inspect the Dataset

```
# Cell 2: Load the Hotel Reviews CSV and inspect its structure
# ──────────────────────────────────────────────────────────────────────────
import pandas as pd

# Load the Kaggle hotel reviews dataset
# Download from: https://www.kaggle.com/datasets/datafiniti/hotel-reviews
CSV_PATH = '../data/hotel_reviews.csv'
df = pd.read_csv(CSV_PATH)

print(f'Dataset shape: {df.shape}')
print(f'Columns: {list(df.columns)}')
print()
print('Sample row:')
print(df.iloc[0])
```

Cell Explanation

Before writing any transformation code, always inspect the raw data. df.shape tells you how many reviews you have (rows) and how many attributes (columns). The column names tell you what fields are available for embedding and indexing. Typical Kaggle hotel review datasets contain: reviews.title, reviews.text, name (hotel name), city, province (state), reviews.date, and reviews.rating. Understanding this schema up front prevents bugs in the transformation step.

Cell 3 — Transform CSV Rows to Individual JSON Files

```
# Cell 3: Convert each CSV row to an individual JSON file
# ──────────────────────────────────────────────────────────────────────────
import os
import json
import uuid

# Directory structure
```

```
TRANSFORMED_DIR = '../data/transformed'
os.makedirs(TRANSFORMED_DIR, exist_ok=True)  # Create dir if not exists

# Column mapping from raw CSV → our clean schema
# Adjust these names to match your actual CSV columns
COLUMN_MAP = {
    'review_title' : 'reviews.title',
    'review_text'  : 'reviews.text',
    'hotel_name'   : 'name',
    'city'         : 'city',
    'hotel_state'  : 'province',
    'date_added'   : 'reviews.date',
}

skipped = 0
for idx, row in df.iterrows():
    # Skip rows with missing review text — these cannot be embedded
    if pd.isna(row.get('reviews.text', '')):
        skipped += 1
        continue

    review = {
        'id'           : str(uuid.uuid4()),   # Unique ID required by Azure AI
Search
        'review_title' : str(row.get('reviews.title', '')).strip(),
        'review_text'  : str(row.get('reviews.text',  '')).strip(),
        'hotel_name'   : str(row.get('name',          '')).strip(),
        'city'         : str(row.get('city',          '')).strip(),
        'hotel_state'  : str(row.get('province',      '')).strip(),
        'date_added'   : str(row.get('reviews.date',  '')).strip(),
    }

    # Write each review as its own JSON file
    filename = f"{TRANSFORMED_DIR}/{review['id']}.json"
    with open(filename, 'w', encoding='utf-8') as f:
        json.dump(review, f, indent=2, ensure_ascii=False)

total = len(df) - skipped
print(f'Transformed {total} reviews → {TRANSFORMED_DIR}/')
print(f'Skipped {skipped} rows with missing review text')
```

Cell Explanation

Several important production decisions are encoded here. First, we use uuid.uuid4() to generate a unique ID for every document. Azure AI Search requires every document to have a unique key field — without this, uploads will fail. Second, we skip rows with missing review_text because an empty string cannot be meaningfully embedded (the resulting vector would be noise). Third, ensure_ascii=False preserves non-ASCII characters (accented letters, Asian scripts) that are common in international hotel reviews — critical for multilingual corpora. Fourth, one file per document (rather than one giant JSON array) makes partial re-processing and incremental updates much easier in production.

4. Step 2 — Creating Embeddings

Embeddings are the heart of semantic search. This step reads each JSON review file, constructs a rich text string combining all relevant fields, sends it to the OpenAI embeddings API, and saves the resulting vector alongside the review metadata. The embedding vector is what Azure AI Search will later use to measure semantic similarity between the review and any user query.

Cell 4 — Initialise OpenAI Client and Define Embedding Function

```
# Cell 4: Set up OpenAI client and embedding utility
# ──────────────────────────────────────────────────────────────────────
from openai import OpenAI
import time


# Initialise the OpenAI client (reads OPENAI_API_KEY from environment
automatically)
openai_client = OpenAI()


def embed_text(text: str, model: str = EMBEDDING_MODEL) -> list[float]:
    """
    Convert a text string into a numerical embedding vector.


    Args:
        text:  The input text to embed (max ~8,192 tokens for text-embedding-3-
small)
        model: The embedding model name


    Returns:
        A list of 1,536 floats representing the semantic meaning of the text
    """
    # Truncate to avoid exceeding the model's token limit
    text = text[:8000]


    response = openai_client.embeddings.create(
        input=text,
        model=model
    )
    return response.data[0].embedding
```

```
def prepare_embedding_str(review_json: dict) -> str:
    """
    Construct a rich, information-dense string from a review JSON object.
    This string is what gets embedded — its quality directly affects retrieval
quality.

    Design principle: include all fields that a user might semantically search
over.
    """
    return (
        f"REVIEW_TITLE: {review_json.get('review_title', '')} "
        f"REVIEW_TEXT: {review_json.get('review_text', '')} "
        f"HOTEL_NAME: {review_json.get('hotel_name', '')} "
        f"HOTEL_CITY: {review_json.get('city', '')} "
        f"HOTEL_STATE: {review_json.get('hotel_state', '')}"
    )


# Quick sanity test
test_str = prepare_embedding_str({'review_title': 'Great View', 'review_text':
'Amazing pool.',
                                  'hotel_name': 'Grand Hyatt', 'city': 'NYC',
'hotel_state': 'NY'})
test_emb = embed_text(test_str)
print(f'Test embedding length: {len(test_emb)} ✓ (expected {EMBEDDING_DIMS})')
```

Cell Explanation

The prepare_embedding_str function is one of the most important design decisions in any RAG pipeline. The string you embed determines what information is encoded in the vector — and therefore what queries can retrieve the document. By concatenating review_title, review_text, hotel_name, city, and hotel_state with explicit field labels (REVIEW_TITLE:, HOTEL_NAME: etc.), we give the embedding model structural context. This means a query like 'beachfront hotels in Miami' can match a review that never mentions 'beachfront' explicitly but does say 'steps from the beach' in the text and lists 'Miami' as the city. The truncation to 8,000 characters is a safety measure — text-embedding-3-small supports up to 8,192 tokens (roughly 32,000 characters), but very long inputs can cause latency spikes and higher costs.

Cell 5 — Generate and Save Embeddings for All Reviews

```
# Cell 5: Generate embeddings for all review JSON files
# ──────────────────────────────────────────────────────────────────────────────
from tqdm import tqdm


EMBEDDED_DIR = '../data/embedded'
```

```
os.makedirs(EMBEDDED_DIR, exist_ok=True)

files = [f for f in os.listdir(TRANSFORMED_DIR) if f.endswith('.json')]
print(f'Found {len(files)} review files to embed')

errors = []

for filename in tqdm(files, desc='Embedding reviews'):
    embedded_path = os.path.join(EMBEDDED_DIR, filename)

    # Skip files already processed (allows resuming interrupted runs)
    if os.path.exists(embedded_path):
        continue

    transformed_path = os.path.join(TRANSFORMED_DIR, filename)

    try:
        with open(transformed_path, 'r', encoding='utf-8') as f:
            review = json.load(f)

        # Build the rich text string and generate embedding
        embedding_input = prepare_embedding_str(review)
        review['embedding'] = embed_text(embedding_input)

        # Save review + embedding to the embedded directory
        with open(embedded_path, 'w', encoding='utf-8') as f:
            json.dump(review, f, indent=2)

        # Rate limiting: OpenAI free tier = 60 RPM
        # Remove or reduce this sleep on paid tier plans
        time.sleep(0.1)

    except Exception as e:
        errors.append({'file': filename, 'error': str(e)})

print(f'\nEmbedding complete!')
print(f'  Processed: {len(files) - len(errors)} files')
print(f'  Errors   : {len(errors)} files')
if errors:
    print('  Error details:', errors[:5])  # Print first 5 errors
```

Cell Explanation

Several production patterns are baked into this cell. The skip-if-exists check (if os.path.exists(embedded_path)) makes the loop idempotent — you can stop and restart it at any time without re-processing completed files or incurring duplicate API costs. The try/except with error collection (rather than raising immediately) means one malformed review file won't abort an entire overnight batch run — errors are logged and the pipeline continues. The time.sleep(0.1) adds a 100ms delay between requests, keeping throughput at ~10 requests/second to stay within OpenAI's rate limits on standard plans. In production on a high-tier plan, replace this with a proper token-bucket rate limiter.

5. Step 3 — Creating the Azure AI Search Index

The search index is the data structure that makes retrieval fast. Think of it as a specialised database schema — it defines exactly which fields exist, what data types they hold, which fields are searchable, which are filterable, and how the vector search should be configured. Getting the schema right is critical: once documents are uploaded, changing the schema requires rebuilding the entire index.

5.1 Provision the Azure AI Search Service (Portal Method)

1. Navigate to portal.azure.com and click 'Create a resource'
2. Search for 'Azure AI Search' and select it
3. Fill in the creation form:

Field	Recommended Value
Subscription	Your active Azure subscription
Resource Group	Same as your AI Vision resource for logical grouping
Service Name	Globally unique name (e.g., 'rag-hotels-search-dev')
Location	East US (same region as your other resources to minimise latency)
Pricing Tier	Basic for development; Standard S1+ required for Semantic Reranker

4. Click 'Review + Create' → 'Create' and wait for deployment (~2 minutes)
5. Navigate to the resource → 'Keys' in the left sidebar → copy 'Primary admin key'
6. Add to your .env file: SERVICE_NAME=your-service-name and SEARCH_ADMIN_KEY=your-key

Cell 6 — Initialise the Search Index Client

```
# Cell 6: Initialise the Azure AI Search clients
# ──────────────────────────────────────────────────────────────────────
from azure.search.documents import SearchClient
from azure.search.documents.indexes import SearchIndexClient
from azure.core.credentials import AzureKeyCredential


credential = AzureKeyCredential(SEARCH_ADMIN_KEY)
```

```
# SearchIndexClient: manages index schema (CREATE, DELETE, LIST indexes)
index_client = SearchIndexClient(
    endpoint=SEARCH_ENDPOINT,
    credential=credential
)

# SearchClient: manages documents (UPLOAD, SEARCH, DELETE documents)
# Note: SearchClient is index-specific — one client per index
search_client = SearchClient(
    endpoint=SEARCH_ENDPOINT,
    index_name=SEARCH_INDEX_NAME,
    credential=credential
)

print('Azure AI Search clients initialised ✓')
print(f'  Endpoint : {SEARCH_ENDPOINT}')
print(f'  Index    : {SEARCH_INDEX_NAME}')
```

Cell Explanation

The Azure AI Search Python SDK provides two distinct client classes for two distinct concerns. SearchIndexClient deals with index administration — creating, updating, listing, and deleting index schemas. SearchClient deals with document operations — uploading documents and executing search queries. This separation follows the Single Responsibility Principle and maps to the two types of API keys Azure provides (admin key for index management, query key for search-only operations). In production, your web application should use a query-only key for the SearchClient, never the admin key.

Cell 7 — Define the Index Schema (Fields + Vector Config)

```
# Cell 7: Define the search index schema
# ──────────────────────────────────────────────────────────────────────
from azure.search.documents.indexes.models import (
    SearchIndex,
    SimpleField,
    SearchableField,
    SearchField,
    SearchFieldDataType,
    VectorSearch,
    HnswAlgorithmConfiguration,
    VectorSearchProfile,
```

```
)

# ── Field Definitions ──────────────────────────────────────────────
# Each field maps to a property in our review JSON documents

VECTOR_SEARCH_PROFILE = f'{SEARCH_INDEX_NAME}_vector_profile'
HNSW_ALGO_NAME        = f'{SEARCH_INDEX_NAME}_hnsw'

fields = [
    # KEY FIELD: Must be unique, non-null, string type
    SimpleField(
        name='id',
        type=SearchFieldDataType.String,
        key=True,   # ← This is the primary key
    ),

    # SEARCHABLE TEXT FIELDS: Included in full-text (BM25) search
    # analyzer_name='en.lucene' applies English-language stemming and stop-word
removal
    SearchableField(
        name='review_text',
        type=SearchFieldDataType.String,
        analyzer_name='en.lucene',
    ),
    SearchableField(
        name='review_title',
        type=SearchFieldDataType.String,
        analyzer_name='en.lucene',
    ),

    # SIMPLE FILTERABLE FIELDS: Can filter/facet but not full-text searched
    SimpleField(
        name='date_added',
        type=SearchFieldDataType.DateTimeOffset,
        filterable=True,
        sortable=True,
    ),
    SimpleField(name='city',        type=SearchFieldDataType.String,
filterable=True),
    SimpleField(name='hotel_name',  type=SearchFieldDataType.String,
filterable=True),
```

```
    SimpleField(name='hotel_state', type=SearchFieldDataType.String,
filterable=True),


    # VECTOR FIELD: Stores the 1536-dimensional embedding
    SearchField(
        name='embedding',
        type=SearchFieldDataType.Collection(SearchFieldDataType.Single),
        vector_search_dimensions=EMBEDDING_DIMS,          # Must match model
output
        vector_search_profile_name=VECTOR_SEARCH_PROFILE, # Links to HNSW config
    ),
]


print(f'Defined {len(fields)} index fields')
```

Cell Explanation

The schema design here encodes several important architectural decisions. SearchableField vs SimpleField is a critical distinction: SearchableField fields are tokenised and indexed for full-text search (BM25 scoring), which is computationally expensive but enables keyword search. SimpleField fields are stored as-is and support filtering, faceting, and sorting — perfect for structured metadata like city or hotel_name. The analyzer_name='en.lucene' on text fields activates English-language processing: stemming ('running' matches 'run'), stop-word removal ('the', 'a', 'is' are ignored), and lowercasing. The vector field uses SearchFieldDataType.Collection(SearchFieldDataType.Single) because a 1,536-dimensional embedding is stored as an array of 1,536 single-precision floats — exactly what this type represents.

Cell 8 — Configure HNSW Vector Search and Create the Index

```
# Cell 8: Configure vector search and deploy the index to Azure
# ──────────────────────────────────────────────────────────────────────────
from azure.search.documents.indexes.models import (
    SemanticConfiguration,
    SemanticSearch,
    SemanticPrioritizedFields,
    SemanticField,
)


# ── HNSW Vector Search Configuration ──────────────────────────────────────
vector_search = VectorSearch(
    algorithms=[
        HnswAlgorithmConfiguration(
            name=HNSW_ALGO_NAME,
            # HNSW tuning (defaults are good for most cases)
```

```
            # parameters=HnswParameters(m=4, ef_construction=400, ef_search=500)
        )
    ],
    profiles=[
        VectorSearchProfile(
            name=VECTOR_SEARCH_PROFILE,
            algorithm_configuration_name=HNSW_ALGO_NAME,
        )
    ]
)

# ── Semantic Reranker Configuration ──────────────────────────────────
# Tell the semantic model which fields carry the most contextual weight
semantic_config = SemanticConfiguration(
    name='ps-hotels-semantic-config',
    prioritized_fields=SemanticPrioritizedFields(
        title_field=SemanticField(field_name='review_title'),      # Most
important
        content_fields=[SemanticField(field_name='review_text')],  # Main body
        keywords_fields=[                                          # Structural
context
            SemanticField(field_name='city'),
            SemanticField(field_name='hotel_name'),
            SemanticField(field_name='hotel_state'),
        ]
    )
)
semantic_search = SemanticSearch(configurations=[semantic_config])

# ── Assemble and Create the Full Index ───────────────────────────────
index = SearchIndex(
    name=SEARCH_INDEX_NAME,
    fields=fields,
    vector_search=vector_search,
    semantic_search=semantic_search,    # Attach semantic reranker config
)

# create_or_update_index is idempotent — safe to re-run
result = index_client.create_or_update_index(index)
print(f'Index created/updated: {result.name} ✓')
print(f'  Fields: {[f.name for f in result.fields]}')
```

Cell Explanation

This cell assembles all three configuration layers into a single SearchIndex object. The VectorSearch configuration registers the HNSW algorithm and links it to a named profile (VECTOR_SEARCH_PROFILE) — this profile name is what the embedding field references in Cell 7, creating the connection. The SemanticConfiguration tells the reranker model which fields to prioritise: title_field is given the highest weight (short, dense signal), content_fields is the main body of text, and keywords_fields provide structured metadata as contextual anchors. The create_or_update_index call is idempotent — if the index already exists with the same schema, it is a no-op; if the schema has changed, it updates the index. This makes the cell safe to re-run during development.

6. Step 4 — Uploading Documents to the Index

With the index schema defined and deployed, the next step is to upload all embedded review documents. Azure AI Search supports batch uploads of up to 1,000 documents per API call. We'll process documents in batches of 1,000 for maximum efficiency.

Cell 9 — Process JSON Files and Upload in Batches

```
# Cell 9: Load embedded JSON files and upload to Azure AI Search in batches
# ──────────────────────────────────────────────────────────────────────────
import glob

EMBEDDED_DIR = '../data/embedded'
BATCH_SIZE   = 1000   # Azure AI Search maximum per upload_documents call

def process_json_file(file_path: str) -> dict | None:
    """
    Load a JSON review file and ensure the embedding is in the correct format.
    Azure AI Search expects embedding as List[float], not nested structures.
    """
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            data = json.load(f)

        # Ensure embedding is a flat list of Python floats (not numpy types)
        if 'embedding' in data:
            data['embedding'] = [float(x) for x in data['embedding']]

        return data
    except Exception as e:
        print(f'  Warning: Could not process {file_path}: {e}')
        return None



# ── Batch Upload Loop ──────────────────────────────────────────────────────
json_files  = glob.glob(os.path.join(EMBEDDED_DIR, '*.json'))
documents   = []
total_uploaded = 0

print(f'Starting upload of {len(json_files)} documents...')
```

```
for i, file_path in enumerate(tqdm(json_files, desc='Uploading')):
    doc = process_json_file(file_path)
    if doc:
        documents.append(doc)

    # Upload when batch is full OR on the final file
    if len(documents) == BATCH_SIZE or (i == len(json_files) - 1 and documents):
        result = search_client.upload_documents(documents=documents)
        succeeded = sum(1 for r in result if r.succeeded)
        total_uploaded += succeeded
        print(f'  Batch uploaded: {succeeded}/{len(documents)} documents
succeeded')
        documents = []  # Clear the batch

print(f'\nTotal documents uploaded: {total_uploaded} ✓')
```

Cell Explanation

The process_json_file function includes a crucial type-safety step: [float(x) for x in data['embedding']]. When embeddings are saved via json.dump and reloaded, they remain as Python floats — but if you ever process them through NumPy (e.g., for normalisation), array elements become numpy.float32 objects. The Azure AI Search SDK's JSON serialiser does not know how to handle numpy scalar types and will raise a TypeError. Explicitly converting to float() eliminates this entire class of bugs. The batch upload loop uses a single boolean condition (len == 1000 OR last file) to flush the batch — ensuring no documents are left unuploaded regardless of whether the total count is evenly divisible by 1,000.

7. Step 5 — Querying the Search Index

The index is populated. Now we implement the three search modalities that Azure AI Search supports, plus a utility function to embed user queries. These functions form the retrieval layer of the RAG pipeline — the component that finds relevant context for the LLM.

Cell 10 — Query Embedding Utility

```
# Cell 10: Embed a user query using the same model used for documents
# ──────────────────────────────────────────────────────────────────────
# CRITICAL: You MUST use the same embedding model for queries and documents.
# Using a different model produces vectors in different spaces — similarity
# scores will be meaningless and retrieval quality will collapse entirely.

def embed_query(query: str) -> list[float]:
    """
    Convert a user's natural-language query into an embedding vector.
    Identical to embed_text() — kept separate for clarity in the pipeline.
    """
    return embed_text(query, model=EMBEDDING_MODEL)

# Test the query embedder
test_query = 'hotels with beautiful ocean view and great breakfast'
test_qvec  = embed_query(test_query)
print(f'Query embedding shape: {len(test_qvec)} dims ✓')
```

Cell Explanation

The warning in the comment is worth emphasising: model symmetry between indexing and querying is the single most important rule in semantic search. If you embed documents with text-embedding-3-small (1,536 dims) but query with text-embedding-ada-002 (1,536 dims by coincidence), the vectors are in geometrically different spaces and nearest-neighbour similarity scores are meaningless garbage. In production, store the embedding model name alongside the index metadata and assert model consistency at startup.

Cell 11 — Full-Text Search (BM25)

```
# Cell 11: Full-Text Search — traditional keyword-based BM25 retrieval
# ──────────────────────────────────────────────────────────────────────
def full_text_search(query: str, top: int = 5) -> list[dict]:
    """
```

```
    Perform a BM25 keyword search over indexed text fields.

    Args:
        query: The user's search string (exact/partial keyword matching)
        top:   Maximum number of results to return

    Returns:
        List of matching documents as dicts
    """
    results = search_client.search(
        search_text=query,           # BM25 searches this string over all
SearchableFields
        top=top,
        select=['hotel_name', 'review_title', 'review_text', 'city',
'hotel_state'],
    )
    return list(results)

# ── Test It ──────────────────────────────────────────────────────────
ft_results = full_text_search('amazing pool facilities')
print(f'Full-text search returned {len(ft_results)} results:')
for r in ft_results:
    print(f"  [{r['hotel_name']} | {r['city']}] {r['review_title']}")
```

Cell Explanation

The search_text parameter activates BM25 retrieval across all fields declared as SearchableField in the schema (review_text and review_title in our case). The select parameter acts like a SQL SELECT — it restricts which fields are returned in each result, reducing network payload and protecting any sensitive fields you may have in the index. Full-text search is fast, cheap, and highly effective for queries with specific known terminology — a user searching 'Marriott breakfast buffet' will reliably find reviews mentioning those exact words. It struggles with paraphrase queries ('places to eat in the morning') where vector search excels.

Cell 12 — Vector Search (Semantic Similarity)

```
# Cell 12: Vector Search — semantic similarity using embeddings
# ──────────────────────────────────────────────────────────────────────
from azure.search.documents.models import VectorizedQuery

def vector_search(query: str, k: int = 3) -> list[dict]:
    """
    Perform a pure vector search — no keyword matching.
```

```
    Finds documents whose embeddings are nearest to the query embedding.

    Args:
        query: Natural language query string
        k:     Number of nearest neighbours to retrieve

    Returns:
        List of semantically similar documents
    """
    # Step 1: Embed the query into the same vector space as the documents
    query_vector = embed_query(query)

    # Step 2: Build the vector query object
    vector_query = VectorizedQuery(
        vector=query_vector,
        k_nearest_neighbors=k,
        fields='embedding',   # Must match the vector field name in the schema
    )

    # Step 3: Execute the search — no search_text means pure vector mode
    results = search_client.search(
        search_text=None,            # ← None disables BM25; pure vector only
        vector_queries=[vector_query],
        select=['hotel_name', 'review_text', 'review_title', 'city'],
    )
    return list(results)

# ── Test It ──────────────────────────────────────────────────────────
vs_results = vector_search('peaceful place to relax with great nature views')
print(f'Vector search returned {len(vs_results)} results:')
for r in vs_results:
    print(f"  [{r['hotel_name']} | {r['city']}]")
    print(f"   {r['review_text'][:120]}...")
```

Cell Explanation

The VectorizedQuery object is the programmatic equivalent of 'find me the k documents whose embeddings are closest to this query embedding in cosine similarity space'. Setting search_text=None and providing only vector_queries runs the search in pure ANN mode — no BM25 scoring, no keyword matching. The k_nearest_neighbors=3 parameter specifies how many nearest documents to retrieve. In production RAG pipelines, a value of 5–10 is common — you want enough context for the LLM to generate a good answer, but too many documents increase token cost and

can dilute the context. The fields='embedding' parameter tells the API which vector field in the schema to search against — critical if your index has multiple vector fields.

Cell 13 — Hybrid Search (BM25 + Vector, Merged with RRF)

```
# Cell 13: Hybrid Search — best of keyword and semantic retrieval
# ──────────────────────────────────────────────────────────────────────
def hybrid_search(query: str, top: int = 5) -> list[dict]:
    """
    Perform hybrid search: simultaneous BM25 + vector search,
    merged using Reciprocal Rank Fusion (RRF).

    This is the recommended approach for production RAG systems.
    It combines the precision of keyword matching with the semantic
    understanding of vector search.

    Args:
        query: Natural language query
        top:   Maximum results to return
    """
    query_vector = embed_query(query)

    vector_query = VectorizedQuery(
        vector=query_vector,
        k_nearest_neighbors=3,
        fields='embedding',
    )

    # KEY DIFFERENCE from vector_search: search_text is now populated
    # Azure AI Search runs BOTH queries and merges with RRF automatically
    results = search_client.search(
        search_text=query,           # ← BM25 component
        vector_queries=[vector_query], # ← Vector component
        select=['id', 'review_text', 'review_title', 'hotel_name', 'city'],
        top=top,
    )
    return list(results)

# ── Test and Compare ──────────────────────────────────────────────────
query = 'quiet hotel with beautiful garden for a relaxing stay'
```

```
print('=== Hybrid Search Results ===')
hs_results = hybrid_search(query)
for i, r in enumerate(hs_results):
    print(f'{i+1}. [{r["hotel_name"]} | {r["city"]}] {r["review_title"]}')
```

Cell Explanation

The elegance of hybrid search is in its simplicity of invocation: the only difference from pure vector search is that search_text is now populated instead of None. Azure AI Search handles the RRF fusion internally — you don't need to implement the ranking merge yourself. Under the hood, BM25 produces a ranked list (doc_1 at rank 1, doc_47 at rank 2, etc.) and the HNSW vector search produces another ranked list. RRF scores each document as 1/(60 + rank) for each list it appears in, sums those scores, and re-ranks. The constant 60 in the denominator dampens the influence of high-ranked results, making the fusion robust to outliers. This is why hybrid search consistently outperforms either method alone in evaluation benchmarks.

8. Step 6 — Semantic Reranking (The Quality Multiplier)

Semantic reranking is the final and most powerful layer of the retrieval pipeline. It re-scores the top results from hybrid search using a deep language model — promoting results that are semantically most relevant to the query's intent, not just its keywords or vector proximity. It also extracts captions and answers from the retrieved documents.

Cell 14 — Semantic Reranker Query

```
# Cell 14: Semantic Search with Reranking, Captions, and Answers
# ──────────────────────────────────────────────────────────────────────
# Requires: Standard tier Azure AI Search (paid feature)
from azure.search.documents.models import (
    QueryType,
    QueryCaptionType,
    QueryAnswerType,
)

def semantic_search(query: str, top: int = 5) -> dict:
    """
    Perform hybrid search with semantic reranking.

    The pipeline:
    1. Hybrid search retrieves top-50 candidates (BM25 + vector)
    2. Semantic reranker re-scores the top-50 using a deep learning model
    3. Returns the top-N most semantically relevant results with captions

    Args:
        query: Natural language query
        top:   Final number of results after reranking

    Returns:
        Dict with 'results', 'answers', 'captions'
    """
    query_vector = embed_query(query)
    vector_query = VectorizedQuery(
        vector=query_vector,
        k_nearest_neighbors=3,
        fields='embedding',
```

```
    )

    results = search_client.search(
        search_text=query,
        vector_queries=[vector_query],
        select=['id', 'review_text', 'review_title', 'hotel_name', 'city'],
        top=top,

        # ── Semantic Reranker Parameters ──────────────────────────
        query_type=QueryType.SEMANTIC,                      # Enable reranker
        semantic_configuration_name='ps-hotels-semantic-config', # Our config
        query_caption=QueryCaptionType.EXTRACTIVE,          # Extract key
snippets
        query_answer=QueryAnswerType.EXTRACTIVE,            # Extract direct
answers
    )

    # Collect results and extract semantic metadata
    docs     = []
    captions = []

    for r in results:
        docs.append(r)
        # Semantic captions highlight the most relevant passage in each doc
        if hasattr(r, '@search.captions') and r['@search.captions']:
            for cap in r['@search.captions']:
                captions.append({
                    'hotel' : r['hotel_name'],
                    'caption': cap.text,
                    'highlights': cap.highlights,
                })

    # Semantic answers are direct answers extracted from any document
    answers = results.get_answers() or []

    return {'results': docs, 'answers': answers, 'captions': captions}



# ── Test the Full Pipeline ──────────────────────────────────────────
query = 'Which hotels are great for families with young children?'
output = semantic_search(query)
```

```
print(f'Query: {query}')
print(f'Results: {len(output["results"])} documents')
print()

if output['answers']:
    print('=== Semantic Answer ===')
    for a in output['answers']:
        print(f'  {a.text}')
    print()

print('=== Top Results with Captions ===')
for i, (doc, cap) in enumerate(zip(output['results'], output['captions'])):
    print(f'{i+1}. {doc["hotel_name"]} ({doc["city"]})')
    print(f'   Caption: {cap["caption"]}')
    print()
```

Cell Explanation

This cell activates the full power of Azure AI Search's semantic pipeline. query_type=QueryType.SEMANTIC is the master switch — it tells the API to route results through the reranker. The semantic_configuration_name must exactly match the name defined in Cell 8 ('ps-hotels-semantic-config'). QueryCaptionType.EXTRACTIVE instructs the model to extract the single most relevant sentence or phrase from each document — extremely valuable for display in a search UI as a result snippet. QueryAnswerType.EXTRACTIVE goes further: if any of the top documents contains a passage that directly answers the query (not just relates to it), the model extracts it as a structured 'answer' object separate from the ranked results. This is similar to Google's featured snippets. The @search.captions field uses Azure's special @ prefix convention for metadata fields returned alongside document fields.

9. Putting It All Together — The RAG Generation Step

The retrieval pipeline is complete. The final step is to connect it to an LLM to generate grounded, cited answers. This is the full RAG loop: retrieve relevant reviews → assemble context → generate response.

Cell 15 — Complete RAG Pipeline Function

```
# Cell 15: Full RAG pipeline — Retrieval + Augmented Prompt + Generation
# ──────────────────────────────────────────────────────────────────────
def rag_answer(user_query: str, top_k: int = 5) -> str:
    """
    End-to-end RAG pipeline:
    1. Retrieve relevant hotel reviews via hybrid + semantic search
    2. Assemble a grounded prompt with the retrieved context
    3. Generate a response using GPT-4o

    Args:
        user_query: The user's natural language question
        top_k:      Number of review documents to include as context

    Returns:
        LLM-generated answer grounded in retrieved reviews
    """
    # ── Step 1: Retrieve ──────────────────────────────────────────────
    search_output = semantic_search(user_query, top=top_k)
    retrieved_docs = search_output['results']

    # ── Step 2: Format Context ────────────────────────────────────────
    context_parts = []
    for i, doc in enumerate(retrieved_docs):
        context_parts.append(
            f"[Review {i+1}]\n"
            f"Hotel: {doc['hotel_name']} ({doc['city']})\n"
            f"Title: {doc['review_title']}\n"
            f"Review: {doc['review_text']}\n"
        )
    context = '\n---\n'.join(context_parts)

    # ── Step 3: Build Prompt ──────────────────────────────────────────
```

```
    system_prompt = (
        'You are a helpful travel advisor for PeerBNB. '
        'Answer the user\'s question based ONLY on the hotel reviews provided. '
        'If the reviews do not contain enough information, say so clearly. '
        'Always cite which hotel (Review number) supports each claim.'
    )

    user_prompt = (
        f'User Question: {user_query}\n\n'
        f'Hotel Reviews:\n{context}\n\n'
        f'Please answer the question based on the reviews above.'
    )

    # ── Step 4: Generate ──────────────────────────────────────────
    response = openai_client.chat.completions.create(
        model='gpt-4o',
        messages=[
            {'role': 'system', 'content': system_prompt},
            {'role': 'user',   'content': user_prompt},
        ],
        temperature=0.3,   # Lower temperature = more factual, less creative
        max_tokens=800,
    )
    return response.choices[0].message.content



# ── Demo the Full Pipeline ────────────────────────────────────────
question = 'Which hotels in New York are recommended for a romantic anniversary
trip?'
print(f'Question: {question}')
print('=' * 60)
answer = rag_answer(question)
print(answer)
```

Cell Explanation

The rag_answer function encapsulates the complete RAG loop and makes several production-quality decisions. The system prompt includes the critical instruction: 'based ONLY on the hotel reviews provided' — this is the grounding constraint that prevents the LLM from falling back on its parametric knowledge and hallucinating hotel recommendations that are not in your data. The 'cite which hotel supports each claim' instruction produces citations, making the response verifiable. Temperature=0.3 is deliberately low for a factual retrieval task — we want the LLM to synthesise the retrieved information faithfully, not exercise creative latitude. The context format ([Review 1], Hotel: ..., Review:

...) gives the LLM clear structure to reason over, improving citation accuracy compared to a raw text dump.

9.1 Quick Reference: Search Method Comparison

Method	When to Use	Code Pattern
Full-Text (BM25)	Known keywords, filter queries, exact hotel names	search_text=query, no vector_queries
Vector Search	Semantic intent, paraphrase, 'vibe' queries	search_text=None, vector_queries=[...]
Hybrid (RRF)	Most production RAG use cases (best default)	search_text=query + vector_queries=[...]
Semantic Reranker	High-stakes queries, customer-facing apps	query_type=QueryType.SEMANTIC + hybrid

Next Chapter Preview

In the next chapter, we take this RAG pipeline to production — deploying it as a FastAPI microservice, adding streaming LLM responses for real-time UI updates, implementing response caching with Azure Redis, and monitoring retrieval quality with automated evaluation using RAGAS (Retrieval Augmented Generation Assessment). We also cover multi-tenant index isolation strategies for enterprise deployments.